

Course Notes — SESSION 8

REASONING ABOUT THE PHYSICAL WORLD

1. Introduction

In our last session, we based intelligence on interacting with and experiencing the physical and cultural world. Such interacting and experiencing arose from moment-to-moment improvising with the body. Representations were useful only when occasional breakdowns arose that required reflection. In this session, we will view breakdowns not as the exception but the rule. Intelligence in AI systems and organizations will involve reflectively reasoning about the physical systems in which AI systems and organizations are situated and embedded to identify and resolve breakdowns. We will refer to such processes as **reasoning about the physical world**.

Foundational AI ideas for reasoning about the physical world emerged in the 1970s and 1980s from students at the MIT AI lab who were proteges of Minsky and his learning in a society paradigm, and were also influenced by McCarthy's ideas about knowledge representation and reasoning. Most of these students were from engineering backgrounds. Engineers typically make use of detailed representations of physical systems (i.e., engineering schematics). In their experience as engineers, these students at MIT found that the complexities of the physical world inevitably make such representations messy and requiring continual modification (or, in the parlance of our previous session, continual “registration” of the physical and cultural world).

A canonical example of reasoning about the physical world in this tradition of AI research is the problem of identifying faults in either the design or repair of an integrated circuit board (ICB). ICBs are devices that control the flow of electrical current in digital or analog machines. There are a vast number of types of ICB functions, and thousands of possible types for each part. Hence, the problem of designing a new ICB, or repairing an existing one with faults, is combinatorially explosive. It cannot be solved just by representing the problem as a search tree or simply (which we have associated with search) or prediction (which we have associated with machine learning) are recast in this tradition as ones of constructing and repairing **causal interpretations** of physical systems that describe how they function.

Overall, the approach is to view such reasoning about the physical world as a type of **constraint propagation**. Unlike our simple Sudoku models, however, the constraint propagation needed here is based on fundamental constraints on the physical world, which we will call “physical laws” or, more commonly, “**first principles**”. Hence, such constraint propagation is sometimes called **first-principles reasoning**. For example, to design or repair an ICB, engineers use electrical engineering “laws”, such as how much voltage can flow between the components, to represent and reason about constraints on variables such as transistors, resistors, and capacitors.

2. Credit assignment as a “debugging” process

2.1 Bugs and debugging

Let us first develop some vocabulary for talking about breakdowns with more of an engineering flavor. A breakdown can also be thought of as a **symptom** of a problem. A symptom can also be called a “**discrepancy**”, or a “**constraint violation**” (i.e., when values of variables in a constraint network do not satisfy a constraint). For our purposes, we will follow the engineering tradition of AI research in this area at MIT and call all such symptoms, discrepancies, or constraint violations as “**bugs**”. Here, bugs are not, as in software programs, merely errors in the syntax of the software code, but rather errors of *registration* (i.e., of aligning representations with the physical world). Hence, we will call intelligence in this session (i.e., learning to assign credit when solving complex problems) as the ability to learn to “**debug**” representations of problems in the physical world.

You can think of debugging as manifesting in two complementary strategies. First, we can assume that our AI system or organization has the correct representation of a problem regarding some physical system, but that there are bugs in the actual system. We will call such debugging “**troubleshooting**”. An example is reconciling a malfunctioning ICB board with a schematic of the board that is known to be correct. Second, we can assume that the physical system is fine, but that bugs may arise in the representation of the system. Examples would be (1) using the wrong schematic to reason about an existing ICB board, or (2) constructing a new schematic to design a new type of ICB board based on the function that we have in mind, where this new schematic will initially go through many rough drafts. We will call these latter debugging processes as “**theory formation**”.

Whether troubleshooting or theory formation, debugging involves an AI system or organization that reasons about the physical world to reconcile “reality” with its representation of it. You can think of such reasoning processes as fundamentally about deriving **function** (what the system does) from the **structure** of it (i.e., from observing the components or parts of the system). Just as Simon’s watch was a metaphor for the structuring of any system, this view of intelligence as *deriving function from the structure of a physical system* is held to be generally applicable, whether to a product, an organization, or a manufacturing process.

2.2 Debugging as a distinct problem-solving process

Recall from our initial sessions of the course that learning to assign credit to solve complex problems can be thought of as a **generate-and-test process**: candidate solutions to problems are generated, and tested for whether they work. A limit of this process, however, is that it assumes that we know the basic shape of the underlying problem — that the candidate solutions are generated within a mostly known search tree or constraint network (i.e., a given performance landscape, with given peaks).

In contrast, the AI idea of debugging proposes that this process also requires reasoning about a physical system to understand it: to understand how function emerges from structure. While there may be some optimal solution to the underlying problem (i.e., some “peak” in a landscape), debugging assumes that the AI system or organization does not know this — it has to construct a representation of the landscape from the ground up. Minsky first pointed out that, rather than reach some optimal peak in a landscape, what is needed is to reach some *threshold of understanding* (what we called earlier in this course a “mesa” landscape).

We will call such a threshold understanding as a **causal interpretation** of the physical system — considered as a representation of meaningful information about its functioning (i.e., not just using Shannon information, and not just describing how the components and parts are assembled but how they produce behavior). This representation is then reasoned about to identify and resolve any “bugs” in it. Hence, the process is really **generate-test-and-debug**, where debugging involves not just generating candidate solutions, but possibly modifying the representation of the problem (and even the goals themselves). Such modifications may involve adding or removing actions or components or re-ordering them based on interpreting how the components causally fit together to produce the system’s functioning.

To see how this interpretative nature of the debugging process is distinct, let us consider how it compares to some of the other problem-solving processes and representational strategies that we have studied.

- **Versus heuristic search:** recall that this idea involved using some set of operators and heuristics to transition from an initial state to a goal state. Yet this assumes that both the operators and heuristics are based on some prior understanding of the global functioning of the system. There is no representation of the physical structure of any system and, hence, no mechanism for reasoning about its functioning. The idea of debugging is that an AI system or organization needs to piece together a tentative representation of this structure and functioning.
- **Versus CSPs:** recall that, in a standard constraint satisfaction problem, the constraints were discrete and precise. For example, in Sudoku, the “all different” constraints had nine discrete values and we could calculate precisely at any state of the game exactly how many remaining possible values for each variable (square of the board) there were. In contrast, the variables in any complex physical system are often continuous and messy. Interpreting its function requires, likewise, a messy and qualitative description of the system.
- **Versus causal nets:** as noted above, debugging involves coming up with some sort of causal interpretation of the functioning of a physical system from its structure. Causal nets, however, assume that such structure can mostly be specified in advance — recall that what is learned in a causal net are the strengths of the probabilities between variables in some given causal structure, rather than the structure itself. Hence, whereas causal nets

are more like a scientist doing a controlled experiment in a lab, debugging is more like an engineer who constructively pieces together an interpretation of how the system fits together from the ground up. Causal interpretations are tentative “theories” that can themselves be “debugged” in place of new theories.

2.3 First principles reasoning

Consider that, even for a physical system such as a mechanical watch with relatively well-defined parts and components, the number of possible “bugs” is combinatorially explosive in ways far beyond even a complex search tree (i.e., state space). Due to this combinatorial explosion, it is not realistic to reason based on acquiring heuristics (one would need practically an infinite number of heuristics) or by representing knowledge about the entire system. Further, even if one somehow *could* represent all the heuristics or other knowledge of a physical system, the messiness of reasoning about the physical world means that an AI system or organization cannot be assumed to be able to select which knowledge is relevant to represent without an extensive process of “debugging” its representations.

Instead, the strategy for assigning credit by debugging is to **reason from first principles** — that is, to use a small number of stable, fundamental “laws” about the physical world that are simple and known to be relevant to a problem. Certain first principles, such as Newton’s laws of physics, seem to hold across a wide (or even universal) set of physical systems. In debugging, the focus is typically on “first principles” specific to some broad class of problems, such as voltage laws from electrical engineering that apply to any ICB circuit. We will call such classes of problems as **domains** and, hence, view debugging as *reasoning about a domain*, as distinct from searching through a state space. Put another way, what remains stable is not a predefined state space (i.e., performance landscape), but rather a much broader domain in the physical world and the “first principles” associated with it.

It follows that, in this view, an AI system or organization should be based on some sort of mechanisms for applying the first principles of a domain to reason about the physical systems underlying the problems to be solved. In the MIT tradition of AI research, this mechanism is commonly thought of as a **domain-specific language**, meaning a language that enables representing the fundamental entities and relations of the domain such that reasoning can be based on the principles of this domain. Such languages can be developed at various levels of abstraction of the same domain. For example, computer-assisted design (CAD) software offers a broad range of engineers a way of reasoning about physical systems based on geometric principles. Nvidia’s CUDA language offers a more specific programming language for designing functions for graphical processing unit (GPU) chips.

3. Debugging strategies

The previous section proposed that, to intelligently solve problems based on reasoning about systems in the physical world, an AI system or organization needs to be able to causally interpret this system and “debug” these tentative causal interpretations. In this section, we will explore some foundational ideas from the AI field for doing so.

3.1 Qualitative reasoning: “envisioning” how a physical system functions

In addition to first principles reasoning, a second — and perhaps counterintuitive — strategy for debugging representations of the physical world is to do so based on what is called “**qualitative reasoning**” with these principles. By qualitative reasoning, what is meant is that information used for reasoning need not (at least initially) contain any reference to precise numbers. As a canonical example from this MIT tradition of AI research, used by alums from Minsky’s AI lab Ken Forbus and Johan de Kleer in their book *Building Problem Solvers*, consider if you have three containers placed at different heights, and connected by pipes. What happens when water is poured into the highest container? It is easy for you to reason that it will flow through the pipes into the second container at medium height, and then into the lowest container. Such reasoning did not require any numbers — just your commonsense about the laws of the physical world (i.e., about gravity and liquids).

This strategy of debugging representations of the physical world based on qualitative reasoning may seem strange to you. In the prior strategy of reasoning based on first principles, what might have come to mind is the possibility of reasoning using formal equations (e.g., Newton’s $F = MA$) for *quantitatively* analyzing a physical system. Surely, one might think, such formality would be preferable. Yet recall that a central premise of the AI ideas that we have discussed here is that solving problems based on reasoning about the physical world is based on messy processes of *debugging* (troubleshooting or theory formation). If an AI system or organization is in the middle of a debugging process, by definition it does not have a precise representation of the physical world, such that quantitative reasoning may initially be of little use!

Let us go back to the container example. How did you know that water would flow from the top one to the bottom one? Surely, this is our commonsense knowledge of how the physical world works, such that all of us have the “first principles” somehow internalized already. Interestingly, de Kleer observed that *even sophisticated engineers* (or other systems designers) initially engage in the same such qualitative reasoning, *even regarding sophisticated systems* (e.g., designing or repairing an ICB). His intuition was that: (1) quantitative analyses enable *optimizing* the performance of a system with *known* functioning; whereas (2) the functioning of the system itself does not permit a quantitative description, and is most fundamentally understood in terms of qualitative language.

What does the *process* of debugging based on such qualitative reasoning look like? De Kleer characterizes this process as one of “**envisioning**” how the components and parts of a physical system function together, which he describes as “**running a qualitative simulation in the mind’s eye**”. More precisely, envisioning means predicting and analyzing changes of the qualitative state of a system (e.g., from water being in one container, to flowing into another). The end states are not goal states, so much as the “logical limits” of the physical system according to the first principles of the domain. You can think of envisioning an entire physical system as akin to a simplified picture of how initial states may unfold into other states (i.e., how various “scenarios” unfold). Causal interpretations arise while doing such envisioning, as principled ways of explaining how a system transitions between states.

3.2 Propagating qualitative information

You can think of qualitative reasoning to envision a physical system as a type of **constraint propagation** process. In our example, the variables would be qualitative variables such as containers or water, and the values could be qualitatively described, such as “full” or “empty”. We will assume that an AI system or organization knows the “domain” of the physical system, in the sense that it has some library of variables that serve as the “building blocks” of the domain. For example, in ICB design or repair, an electrical engineer reasons based on some library of building block schematics and parts (e.g., transistors, resistors, and capacitors).

The constraints on these variables, however, are represented not just as allowable values (as in Sudoku), but as **causal interpretations** on how variables are linked. Note, again, that the causal nets that we studied earlier in the course are only specific ways of talking about such causal interpretations that are possible only when we can treat things as if they are a controlled experimental setting. For a typically messy representation of a real-world physical system to be “debugged”, the constraints are considered as variables that mutually influence one another — or what De Kleer calls “**confluences**” — that reflect the first principles of the domain. For example, the height differences of the water containers influence how fast water flows from one container to another, as does the width of the pipes connecting them.

What you should see here is that the “magic of propagation” that we talked of earlier becomes a much more iterative and interpretative process than was implied by our Sudoku example — “debugging” involves understanding and reasoning about how various influences on the behavior of a physical system come together in complex ways. Here, it is more useful to think of the constraints (i.e., confluences) as not just tests or target ranges, but rather as populated by agents, i.e., “**propagators**” in an organization. The propagators in this “organization” qualitatively represent and reason about the constraints as a basis for deciding which information to propagate to other constraints and, hence, to other propagators, for constructing and informing one another’s causal interpretations of local behavior. Next, we will look at how reasoning in this “organizational” context involves propagating information not just about the constraints and

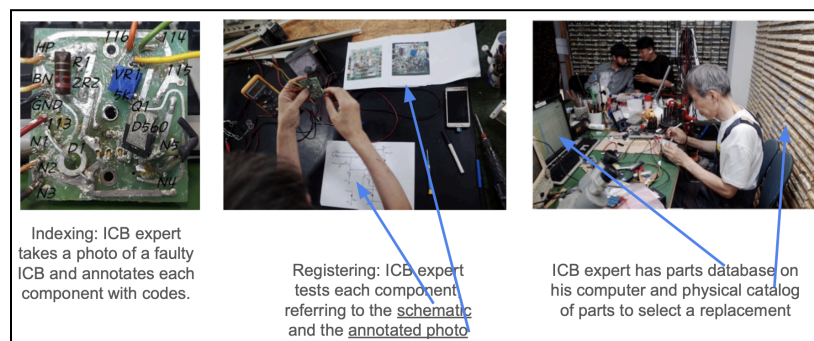
variables themselves, but about tentative assumptions made about them in a complementary representation to the constraint network called a **dependency network**.

3.3 “Truth maintenance” using dependency networks

In the previous section, we framed the debugging process as carried out by an organization of “propagators”. Each propagator processes information about constraints on some physical system (“confluences”). They propagate information to one another to qualitatively reason about and causally interpret (i.e., to “envision”) the behavior of this physical system. Given the messiness of debugging, however, this constraint propagation process is not as straightforward as the Sudoku example that we looked at earlier in the course. In the Sudoku example, we could just store information on the remaining possible values (1-9) for each of the squares of the board (81). Hence, we just had to have a theoretical maximum “memory” of 729 possible values (81x9, assuming no predefined squares), as in the figure below.

7	123	123	123	123	123	123	123	123	1
456	456	456	456	456	456	456	456	456	
789	789	789	789	789	789	789	789	789	
123	123			123			123	123	
456	456	6	5	456	9	8	456	456	
789	789			789			789	789	
123		9	4	123	2	3	7	123	
456	456			456	456		456	456	
789	789			789	789		789	789	
123	7	123		123			123	123	
456	456		4	456	3		456	456	
789	789			789			789	789	
123	123		3	123	123		123	123	
456	456			456	456	7	456	456	
789	789			789	789		789	789	
123		6	123	123	7	1	123	123	
456	456		456	456			456	456	
789	789		789	789			789	789	
123	4	8	123	123			123	123	
456	456		456	456	9	6	456	456	
789	789		789	789			789	789	
123	123		7	123	8	1	123	123	
456	456			456			456	456	
789	789			789			789	789	
123		3	123	123			123	123	
456	456		456	456	456		456	456	
789	789		789	789	789		789	789	
									2

In contrast, as described in the previous section, debugging representations in the physical world depends on qualitative reasoning, such that the “memory” of an AI system or organization should represent diverse qualitative information or knowledge about a physical system. As depicted in the photos below, for example, repairing an ICB board might involve annotating the components of the board with codes (left), comparing the board to various schematics (middle), and selecting replacement parts using a database of 1000’s of parts with rich descriptions of their functions and specifications.



This need for qualitative representation and reasoning brings in all the issues raised by McCarthy in our earlier session on knowledge representation and reasoning — the **frame problem**, and the need to solve this problem based on common sense; and the need for expressive languages that enable **elaboration tolerance** in the representations. We will confine our focus here, however, to the nature of the constraint propagation process — i.e., of how “debugging” requires a distinct way of inducing the “magic of propagation”.

The key idea, pioneered by Minsky’s student **Gerald Sussman** and his colleagues in the 1970s, is that the **constraint network** needs to be supplemented by what is called in this tradition of AI research a **dependency network** — a locally-stored (i.e., propagator- or constraint-level) memory of all the qualitative information and knowledge representations to make them available to the propagators. Think of a dependency network as, for example, a sort of systematically recorded and continually-updated memory of all the annotations relevant to each of the components of the ICB in the images above. Informally, you can think of it as sort of a “scratchpad” or “whiteboard” maintained by each propagator to keep track of information and knowledge during the debugging process.

The key idea of a dependency network is to enable debugging as a process of making and modifying (or retracting) **tentative assumptions** about the inevitably partial, messy and often conflicting qualitative representations of a physical system — referred to as debugging based on “**truth maintenance**”. Hence, a dependency network is part of what is called a “**TMS**”, or truth maintenance system. The key idea of a TMS is that, in addition to constraint propagation, information processing involves a parallel activity of tracking interdependencies among tentative knowledge representations and their assumptions. In this network, the nodes are qualitative representations of knowledge (assertions, assumptions, contradictions, rules, etc.). The links are beliefs called **justifications**, which represent things like consequences, antecedents, or explanations, and labels that indicate whether such justifications are “in” or “out” (i.e., whether they are tentatively believed to be true or not).

The key idea is that this TMS and dependency network dramatically (1) enhances the “magic of propagation” by enabling qualitative reasoning; and also (2) greatly enhances the efficiency of any search processes used. The key idea is that reasoning about the physical world becomes “**dependency-directed**” — informed by rich and diverse information and knowledge about interdependencies (constraints) that enables *explaining* assumptions, tracking *provenance* (i.e., which propagator made the assumption), handle or resolve inconsistencies in assumptions, and store all the results of such reasoning for further use. Put another way, when “bugs” arise, an AI system or organization can trace the bugs to specific assumptions in the constraint network, “debug” the local representation based on reasoning about these assumptions using rich representations, and debug the constraint network itself by propagating the results of such rich reasoning.

Overall, the intuition can be seen by contrast to the earlier idea in search of Simon's means-ends **analysis**. In that idea, problem-solving was less dependency-directed than it was **chronological** — we assumed that we could just use operators and heuristics to transition from an initial state to states closer and closer to a goal state. In contrast, the idea of debugging based on constraint propagation using dependency networks is problem-solving as a messy, iterative process in which assumptions (even about the nature of the goal state) may need to be continually revised and explained. Seen this way, “bugs” are not so much something to avoid (akin to taking a wrong path in search that ends up “stuck” on a local peak in a performance landscape), as an inevitable and helpful basis for maintaining tentative beliefs (“truths”) until one has some threshold understanding of a physical system.

4. Approaches to reasoning about the physical world

As in our previous sessions, we will understand AI ideas for reasoning about the physical world in terms of two paradigmatically-different approaches. One approach, developed mostly outside of the MIT tradition described above but which is currently popular, is based on **deep learning** (i.e., Boltzmann machines) and is commonly referred to today as the “**world models**” approach to AI. Another approach, which Gerald Sussman calls simply the “**propagator model**”, derives from this MIT tradition and can be thought of as a distillation of ideas described above.

4.1 Machine learning-based “world models”

“World models” refer to several recent deep learning-based approaches for representing and making predictions about the physical world. These approaches typically extract patterns from large volumes of data that are structured in ways consistent with Shannon information (i.e., as meaningless bitstrings such as vector embeddings). The goal of world models is to use these patterns to predict the next state of some physical world that results from an action taken in this world. Hence, rather than reason based on first principles about the physical world, the strategy is to indirectly “learn” such principles from Shannon-style data (e.g., sensor and image data stored in bit string-like structures) about this world.

World models are commonly compared, and proposed as an alternative, to GenAI programs such as LLMs. Whereas LLMs predict the next word in a sentence, or whereas image generators predict the next pixel in an image, world models are designed to predict, e.g., broader scenes within an image or broad outcomes of actions in the physical world. This broadness is needed since actions in the physical world are considered vastly more complex (in a Shannon information sense) than actions to manipulate words. Due to this complexity, information must be processed at a higher level of abstraction (e.g., areas in an image, rather than individual pixels) to avoid combinatorial explosion. Hence, the strategy is, for instance, to learn our earlier container example — that pouring water in the high container will flow through the pipes to the lower containers — by extracting patterns from vast amounts of visual data (images, as well as

videos or 3D environments) about objects such as containers and water, their positioning, and their interdependencies, again stored as Shannon information. Hence, such world models essentially learn representations of regularities in the physical world as mappings of actions to bit string-like structures (embeddings) about higher-level objects in this world. They “reason” about the world indirectly, by taking in what is called in machine learning some “context” (e.g., a prompt from a user, or sensor data from the physical world at the moment) to predict the consequences of various actions and select among these actions.

World models have shown impressive performance in many contexts, including self-driving vehicles. On the other hand, compared to our discussion in the first three sections of this session, world models do not represent knowledge in ways that can be qualitatively reasoned about, nor do they allow for using a dependency network to precisely trace and explain bugs, nor are they elaboration tolerant in the way that McCarthy had in mind and that the MIT tradition of research in this area would emphasize (i.e., there is no clear way to incrementally add to and debug information and knowledge to the dependency network itself). Further, world models require vastly more processing capacity, making their scalability possibly infeasible, or at least undesirable, in the type of real-world organizational settings emphasized in the MIT tradition of research about reasoning about the physical world.

4.2 Sussman’s propagator model

Another approach, pioneered by MIT AI researcher and Minsky’s student Gerald Sussman, is what he has called the **propagator model**. This model is based on the building blocks of: (1) “propagators” in a constraint network, who work with (2) “cells”, or stores of memory about tentative beliefs, who (3) reason about this memory opportunistically, meaning based on whatever information has been propagated at any particular time. You can think of this propagator model as a distillation of the approach to reasoning about the physical world that we have explored in the first three sections of this session’s course notes.

Designing AI systems or organizations in terms of propagators does not require extracting patterns from vast volumes of bit string-like data. Rather, the idea of propagators is that they can serve as a general model to support debugging processes — of continually constructing beliefs about tentative causal interpretations of physical systems and “maintaining” an accurate representation of these beliefs as propagation unfolds. Hence, the goal is not just (or primarily) to predict the next state of the world per se, so much as to support the process of piecing together tentative representations of how a physical system functions.

To see how the propagator model differs from world models (or LLMs) more fundamentally, however, it will first help to compare it to the *very idea of programming itself*, as captured by the example of the Turing machine from earlier in the course. Recall that we viewed the Turing machine as a general way of thinking about plans or programs for solving problems — as

generating sequences of actions for transitioning from some initial state to a goal state. This Turing machine-style view of AI systems (and programming in general), however, assumed a *single computer* with a *consistent view* of the world, and where actions are *sequential* in nature. In contrast, the propagator model offers a view of planning or programming that is (1) potentially far more **expressive** (i.e., can reason about rich qualitative information and knowledge and *dependency networks*) and, hence, which can potentially be far more **elaboration-tolerant**; and (2) which is organized around constraint networks that are spatial rather than sequential, such that problem-solving actions can unfold in any direction, which Sussman emphasizes is far more **robust** and, hence, suitable for the debugging processes central to effectively reasoning about the physical world; and (3) where the “propagators” in this network *could be anything* that can store and process information.

It is this final point that should highlight for you the paradigmatic difference with machine learning-based world models. Sussman’s argument here is that, while it may be tempting to think that extracting bit string-like patterns is the most general and powerful basis for reasoning about the physical world, this is a trap. In fact, in his view, doing so is quite narrow — one can only really propagate information in ways similar to the propagation mechanisms of a Boltzmann machine (i.e., where the propagators keep track of weights between the bit strings). In contrast, his idea of the propagator model is that the basic building blocks for reasoning about the physical world *could be anything* (CSPs, causal nets, search trees, or deep learning or LLMs or world models), and organized to propagate information in any order. It is this deeper generality, in his view, that is needed to support the most flexible debugging processes for interpreting and taking action in complex physical worlds.